159102 workshop – Week 8

1. Run the program in the notes that uses the recursive factorial function. Enter the number 24. Explain what happens.

2. In the program that uses the recursive factorial function, modify the function as follows:

```
int factorial(int x) {
  int result;
/**
  if (x == 0) {
    return 1;    // the base case
  }
**/
  cout << x << ", ";
  result = x * factorial(x - 1);
  return result;
}
```

Now run the program for a value of 5. Explain what happens.
Basically, we removed the base case. This should cause a stack overflow but some systems look out for this and stop the program before the stack overflow occurs.

3. You have won first place in a competition and you now get to select 2 prizes from a group of 6 prizes. How many different combinations of 2 prizes could you possibly select?
We use the formula `C(n, r)` where `C` stands for "Combinations", `n` is the total number of items and `r` is the number of items you wish to select. The actual formula is:

```
C(n, r) = n! / (r! * (n – r)!)
```

Write a program that uses this formula (along with the recursive factorial function) to calculate how many different combinations of 2 prizes are possible with a total number of 6 prizes.
The correct answer is 15

4. There are two ways to compare the speed of algorithms. One way is to time the program but this is not always accurate. A program may pause if the system needs to do urgent work, e.g. a security check. Another way to compare algorithms is to count important operations. For example, when looking at search algorithms, **the important operations are the comparisons**. In other words, how often do we need to ask "is this item in the array the one that I am searching for?".

Assume an array of 10,000 numbers in order from 1 to 10,000. How many comparisons are required to find the number 2,500 if:

a) you are using linear search?

b) you are using binary search?

5. Design your own system of caves and prepare to use the caves program from the notes. You will need to do the following:

1. Draw a map of some caves with doors that lead to other caves (similar to the map in the notes).

2. Write the `loadalldata` function for your system of caves.

3. Run the program.